Wire to Ring Zero

The Remote Kernel Attack Surface That Years of Fuzzing Never Reached

Black Hat USA White Paper

Has the Linux kernel network stack really been thoroughly tested? For years, distributed fuzzing behemoths like Syzkaller have hammered the kernel around the clock, churning out thousands of CVEs. It is easy to look at this relentless effort and assume the remote attack surface has been hardened. It has not. Despite millions of CPU hours spent, Syzkaller and Google's Syzbot have found **zero** remotely triggerable vulnerabilities in the same period we found **15**. This is not a resource problem. It is a fundamental tooling gap — and this paper explains why, and what we built to close it: **NetPanic**, a prototype fuzzer purpose-built for the remote kernel attack surface.

---

1. The Forgotten Attack Surface

The Linux kernel implements the full network protocol stack, from Ethernet framing and IP routing through TCP state machines to application-layer services like NFS and SMB. Every byte of every inbound packet is parsed and processed at ring 0. A single exploitable bug in this path hands a remote adversary a kernel-privilege primitive — with no local access, no foothold, no user account on the target. Just packets over the wire.

Yet the kernel security community has spent over a decade focused almost exclusively on the *local* attack surface: the post-breach scenario where an attacker already has code execution and tries to escalate privileges via system calls. This model has been enormously productive for local privilege escalation research, but it leaves the remote attack surface effectively unaudited.

The asymmetry is stark. Organizations watch years of continuous Syzbot runs reporting no remote vulnerabilities and conclude the network stack is hardened. Our work shows this conclusion is wrong — the bugs were invisible to the tools.

2. Two Barriers That Block Existing Tools

We identified two fundamental technical barriers that prevent every existing fuzzer from reaching this attack surface.

2.1 Barrier 1: Extreme Input Complexity

The kernel network stack does not process flat byte sequences. It expects fully encapsulated, multi-layer packets:

```
[ Ethernet Header
  [ IP Header (v4/v6)
    [ TCP/UDP Header
      [ Application Payload (NFS, SMB, ...) ] ] ] ]
```

A fuzzer must generate packets valid enough to pass initial sanity checks (correct checksums, valid header lengths, proper encapsulation) while also malformed enough to trigger bugs in deep protocol logic. Too conservative and it exercises only happy paths; too aggressive and the kernel silently discards packets at the earliest parsing stage.

Why kernel fuzzers fail here. Syzkaller's description language (Syzlang) was designed to model system call arguments, not deeply nested multi-layer network packets. In our head-to-head evaluation, a Syzkaller

instance tuned exclusively for network operations *failed to complete a single TCP handshake* over 24 hours. Without completing a handshake, it cannot touch any TCP state machine logic, let alone NFS or SMB.

Why protocol fuzzers fail here. Tools like AFLNet and StateAFL target user-space protocol implementations. They are fundamentally incompatible with kernel-level testing: they cannot collect kernel code coverage via KCOV, detect kernel crashes, or drive the kernel's network stack directly.

2.2 Barrier 2: Dual-Channel Input Coordination

Unlike most kernel subsystems that accept a single input type, the network stack requires tight, ordered coordination between two coupled channels:

1. **Network packets** arriving from a network interface.
2. **System calls** from user-space (`socket()`, `bind()`, `listen()`, `accept()`).

These channels are interdependent in precise ways. The kernel silently drops every incoming TCP SYN unless a process has already called `listen()` on the appropriate socket. An NFS request is discarded unless the NFS server daemon is running and its RPC programs are registered. A fuzzer cannot just inject packets — it must ensure the right system calls have executed in the right order at the right time, for every protocol it targets.

This requirement is what causes Syzkaller's throughput to collapse to near zero: its generated system call sequences simply do not establish the preconditions the kernel needs to process the injected packets.

3. NetPanic and the Fuzzer-in-the-Middle Architecture

NetPanic is our prototype implementation of a new fuzzing methodology for the remote kernel attack surface. Its central design principle is the **Fuzzer-in-the-Middle (FitM)** architecture, which draws an analogy from the well-known Man-in-the-Middle attack concept.

3.1 Core Insight

Instead of constructing packets from scratch — which requires solving both barriers above from the ground up — NetPanic positions the fuzzer *between* two real kernel network stack instances that are communicating via real user-space programs.

```
[ Kernel Stack A (Server) ] <-- FitM --> [ Kernel Stack B (Client) ]
         |                  intercept                  |
         |                  & mutate                   |
         +-------------- Fuzzing Manager ---------------+
```

This single architectural decision resolves both barriers simultaneously:

- **Input complexity:** The two real kernels generate perfectly valid, fully encapsulated packets themselves. The fuzzer does not need a grammar or protocol specification — it inherits the kernel's own expertise in packet construction for free.
- **Dual-channel coordination:** The user-space client and server programs naturally invoke the correct system calls in the correct order. A TCP server calls `listen()` before the client calls `connect()`; an NFS server registers its RPC programs before the client mounts. Packet-syscall synchronization emerges as a *free property* of the architecture.

In practice, the fuzzer operates as a live interception layer on every packet that flows between the two stacks. Because the two kernels are engaged in real protocol communication, every captured frame is a fully encapsulated, semantically valid packet: it has passed the generating kernel's own protocol logic and carries correct checksums, valid header lengths, and well-formed application-layer payloads. The fuzzer then introduces controlled malformation to selected packets before forwarding them: targeted mutations at the session, packet, or field level that corrupt precisely the target semantic unit while leaving the surrounding structure intact. The receiving kernel, which has already reached the correct state via the legitimate prior exchange, cannot discard the packet at an early sanity-check stage and is instead forced to advance its state machine into the deeper parsing and processing logic where vulnerabilities reside. This is the key operational advantage of the FitM position: the fuzzer inherits the kernel's own packet-construction expertise, then

exploits it by turning the session's structural validity into a vehicle for delivering malformed inputs deep into unaudited code paths.

3.2 System Architecture

NetPanic uses a **manager-agent** split between a host process and a guest VM.

Fuzzing Manager (host). The manager is the fuzzing brain. It maintains the seed corpus, drives the mutation engine, collects and interprets KCOV coverage observations, schedules seeds for re-mutation based on derivation depth and throughput, and detects crashes from kernel log output. It communicates with the agent over a lightweight RPC channel, sending seeds and receiving observation tuples $\langle C, S, L, T \rangle$ after each session. All state including the corpus, coverage map, and crash reports are persisted on the host and survives VM resets.

Fuzzing Agent (guest VM). The agent runs inside a QEMU virtual machine as a thin, stateless execution harness. Its only jobs are to execute a session as directed, intercept and mutate packets as specified by the modifier set, collect KCOV traces, and return observations to the manager. The VM runs the target Linux kernel compiled with both **KCOV** (basic block coverage tracing) and **KASAN** (kernel address sanitizer for memory error detection), so that memory corruption bugs manifest as kernel panics rather than silent memory scribbles.

Network stack isolation. Inside the VM, the agent uses Linux **network namespaces** to create two completely isolated kernel network stack instances. Each namespace has its own routing tables, socket state, and protocol state machines and they share nothing. Each stack is connected to a **TAP virtual device** as its sole network interface. TAP operates at Layer 2 (Ethernet), giving the agent complete read/write access to raw Ethernet frames including all headers. All traffic between the two stacks is routed through the agent's interception layer, which sits logically in the middle and has full visibility and control over every frame.

A key property of TAP is that `write()` calls are processed *synchronously* by the kernel's packet receive path, bypassing the deferred processing queues used by physical NICs. This means KCOV coverage collected immediately after a `write()` is precisely attributable to that single injected packet, with no contamination from background kernel activity.

3.3 Seeds and Session Drivers

A *seed* in NetPanic is a pair Seed = (D, M), where D is a pair of **Session Drivers** and M is a set of **Packet Modifiers**.

Session Drivers are ordinary user-space programs that establish a valid client-server communication session. To fuzz NFS, the drivers are a standard NFSD kernel server and client performing file operations. To fuzz SMB, they are the ksmbd server and an SMB client. No protocol grammar is needed: the kernels generate all the packets; the programs provide correct system call sequencing. This makes NetPanic **protocol-agnostic** and supporting a new protocol requires only writing a client/server program pair.

Packet Modifiers are declarative mutation instructions of the form $\langle \text{idx, op, meta} \rangle$, targeting a specific packet in the session by index with a specific mutation operator. The session drivers produce a clean ordered packet sequence $[P_1, P_2, \ldots, P_k]$; modifiers are then applied to specific packets before they reach the destination kernel. In-flight packets are dissected using Wireshark's protocol analysis engine, giving the mutator a structured, field-level view of every header.

3.4 Three-Level Structure-Aware Mutation

NetPanic's mutations operate at three hierarchical levels. Every mutation targets a semantically meaningful unit rather than raw bytes.

Session level — manipulate the sequence and timing of packets:

- `DropPacket`: Remove the packet from the session entirely.
- `DelayPacket`: Hold the packet for a specified duration before forwarding.

Packet level — modify the protocol layer composition of a single packet:

- `ReplaceProtocol`: Swap one protocol layer for another.
- `AddProtocol` / `RMProtocol`: Insert or remove a protocol layer.

Field level — target individual fields within protocol headers (informed by Wireshark dissection):

- `SetToExtremeL` / `SetToExtremeS`: Set field to minimum or maximum boundary value.
- `SetToRandom`: Assign a uniformly random value.
- `ArithmeticPlus` / `ArithmeticMinus`: Increment or decrement by a small delta.
- `AddOptionalHeader` / `RMOptionalHeader`: Add or remove optional headers (TCP options, IPv6 extension headers).

Two heuristics keep the search tractable. **Sparsity** keeps the modifier set small per seed: network sessions are fragile, and overly aggressive mutation in an early packet terminates the session before the fuzzer reaches deep protocol states. **Exclusivity** prevents packet-level and field-level operators from being applied in the same pass, since a layer replacement invalidates any field-level targets from the original structure.

3.5 Coverage-Guided Evolution

After each session, the agent returns an observation tuple $\langle C, S, L, T \rangle$: KCOV basic block coverage, the full session packet trace, kernel and user-space logs, and execution time. A seed is marked *interesting* and added to the corpus if it increases coverage, produces a previously unseen non-fatal kernel log message, or generates a session with a new packet count.

The evolutionary process mutates only the Packet Modifier set M; Session Drivers remain fixed. Over successive generations, modifiers drift toward mutations that probe increasingly deep and complex kernel code paths. The scheduler prioritizes seeds by derivation depth and deprioritizes seeds that consistently time out.

4. Results

4.1 Vulnerability Discovery

In a single 24-hour campaign against Linux kernel 6.14 (with KCOV and KASAN), NetPanic discovered **15 remotely triggerable vulnerabilities** spanning TCP, SMB (v2/v3), NFS (v3/v4), and SUNRPC. The same Syzkaller baseline, given equivalent hardware and run for a **3-month campaign**, found **zero** novel remotely triggerable vulnerabilities and could not reproduce any of NetPanic's findings.

#	Impact	Class	Protocol	Status
1	Kernel Panic	Null-Ptr Deref	SUNRPC	Patched (CVE-2025-38089)
2	Kernel Panic	Use-After-Free	SUNRPC	Patched (CVE-2025-38501)
3	Kernel Panic	Memory Drain	NFSv4	Patched (CVE-2025-40210)
4	Kernel Panic	Use-After-Free	NFSv2/v3	Patched (CVE-2025-40212)
5	System Hang	Infinite Loop	SMBv3	Patched (CVE-2026-23220)
6–15	Panic/Hang/Perf	Various	SMB/NFS/TCP	Under disclosure

Bug classes: 6 use-after-free, 2 null-pointer dereference, 2 memory drain, 1 general protection fault, 1 infinite loop, 3 hang/degradation. The prevalence of UAF is particularly significant: remote UAF in ring 0 is among the highest-value primitives in the kernel exploit threat landscape.

4.2 Coverage and Throughput

Metric	NetPanic	Syzkaller
TCP code coverage	∼400% more	Baseline
TCP handshake completion	Consistent	0 over 24 h
Packet injection throughput	>100×	Near-zero
NFS code coverage	Substantial	Zero

The table above distills a stark gap. The most structurally telling entry is TCP handshake completion: Syzkaller completes zero handshakes over a full 24-hour run, because its generated packets are structurally invalid and are silently discarded by the kernel's earliest parsing stage. Without a completed handshake, the entire TCP state machine is unreachable, and every protocol that runs over TCP is equally out of reach. This explains why Syzkaller's NFS and SMB coverage entries read zero: it is not that these subsystems are small or easy to miss, but that the tool never gets past the transport layer at all.

NetPanic's overall coverage figure (∼400% more than Syzkaller) reflects access to deep state machine paths that Syzkaller's packets never reach. Packet injection throughput tells a related story: NetPanic sustains over 100× Syzkaller's rate because it does not waste iterations on packets that are dropped immediately. Every injected packet is structurally valid enough to be accepted and processed, so each fuzzing iteration exercises real kernel logic rather than being silently discarded at the door.

5. Two Illustrative Bugs

CVE-2025-38089 (Remote Null-Pointer Dereference via `SVC_GARBAGE`). The SUNRPC server dispatch loop handles three outcomes from `svc_authenticate()`: `SVC_OK` (proceed normally), `AUTH_ERROR` (reject with an RFC-compliant auth reply), and `SVC_GARBAGE` (credential decoding failed — send a `GARBAGE_ARGS` reply). The `GARBAGE_ARGS` path calls `svc_send()`, which writes the RPC accept status through `rqstp->rq_accept_statp`. The bug is that `rq_accept_statp` is initialized only in the `SVC_OK` and `AUTH_ERROR` branches; the `GARBAGE_ARGS` branch never sets it. On a fresh kernel thread that has not yet processed any RPC, the pointer is `NULL`, and the dereference produces an immediate kernel panic. On a reused thread it retains a stale value from the previous request, turning the write into an arbitrary memory scribble. No credentials are required: a single malformed packet that causes RPC credential decoding to return `SVC_GARBAGE` is sufficient. The fix routes `SVC_GARBAGE` through the `AUTH_ERROR` path with reason `AUTH_BADCRED`, as specified by RFC 5531 §7.2, sidestepping `rq_accept_statp` entirely.

CVE-2026-23220 (Infinite Loop via Compound SMB2 Signature Failure). The ksmbd in-kernel SMB server processes compound SMB2 requests which contains multiple chained commands in function `__process_request()`. The field `work->next_smb2_rcv_hdr_off` holds the byte offset from the current command header to the next one in the chain; the processing loop advances by this value after each command. When signature verification fails on a signed SMB2 request, the handler calls `set_smb2_rsp_status(work, STATUS_ACCESS_DENIED)` to prepare an error response. This function resets `next_smb2_rcv_hdr_off` to zero as part of clearing the response buffer. The handler then returns `SERVER_HANDLER_CONTINUE`, signaling the loop to proceed to the next chained command. With the offset at zero, however, `is_chained_smb2_message()` re-examines the same header location; since the `NextCommand` field in a well-formed compound packet is non-zero, the function reports another chained message exists, and the loop re-enters the same already-failed command. The result is a CPU-pinned infinite loop that floods the kernel log with repeated signature-error messages and renders the SMB server permanently unresponsive until the machine is rebooted, with no memory corruption involved. The fix is a one-line change: return `SERVER_HANDLER_ABORT` instead of `SERVER_HANDLER_CONTINUE` after a signature validation failure, terminating the loop before the zeroed offset sends it into a cycle.

These two bugs illustrate the breadth of the remote attack surface. CVE-2025-38089 is pre-session, requires no credentials, and on a first-touch kernel thread produces an immediate panic — the most direct form of remote kernel crash. CVE-2026-23220 is a different class of bug entirely: not memory corruption but a liveness failure, reachable only by a tool that can generate correctly structured, signed SMB2 compound frames. Both were invisible to any fuzzer that cannot complete a session.

6. Conclusion and Future Directions

The Linux kernel's remote attack surface has been critically under-examined: not because it is safe, but because the tools the community relies on are architecturally incapable of reaching it. Syzkaller and Syzbot have run continuously for years and found zero remotely triggerable vulnerabilities; NetPanic found 15 in a single 24-hour campaign. The difference is not compute power. It is the right threat model and the right architecture to match it.

The core lesson of this work is that the absence of reported remote kernel vulnerabilities is not evidence of security, but the evidence of a tooling blind spot. Any organization running Linux with NFS, SMB, or SUNRPC services exposed to the network has been operating with an effectively unaudited attack surface. The prevalence of the vulnerabilities discovered by NetPanic is a particular concern, since remote UAF in ring 0 is a high-value primitive for kernel exploit development, not merely a denial-of-service primitive.

FitM as a methodology is not Linux-specific. The two barriers we identified (extreme input complexity and

dual-channel coordination) apply to any OS kernel network stack. We have begun porting NetPanic to the **Windows kernel** and **ChromeOS**, and expect the same class of bugs to surface there. The approach is also extensible: co-mutating system call parameters alongside packets is a natural next step that the current design deliberately holds constant.

NetPanic will be released as open source at the time of the Black Hat briefing. The intent is to give the security community a practical starting point for auditing their own kernel deployments, from the attacker's true vantage point: the network.